// QE LEARNING PATH · PHASE P03 · ~17 H

pytest for QE

Six tasks that take you from your first test function to a plugin of your own: discovery, **debugging failures**, data-driven tests, fixtures, markers and hooks. The test examples here **run real pytest in your browser**, against your P02 code. Pick the flags next to the Run button, read the output, then answer the interview questions out loud.

- 🧠 **Level 1 — Crack It** · the 🟢 low example
- 🚀 **Level 2 — Impress Them** · the 🔵 medium example
- 🏆 **Level 3 — Hero Mode** · the 🟣 high/complex example

🟢 Zero · 🔵 Hero · 🟣 Super Hero task levels

Z · H · SH interview-question levels

+ ADDED not in P03-pytest.md — added to fill a gap

P03 — THE PHASE AT A GLANCE

🗺️ Six tasks, ~17 hours

Goal: master the test runner every Python QE job uses: structure, **debugging failures**, data-driven tests, fixtures, markers, CLI options and plugins. **Apps:** your P02 code, SauceDemo data, Restful-Booker. **Prerequisites:** P02. **Time:** ~17 h.

ID	TASK	LEVEL	TIME	YOU DELIVER
P03-01	**First pytest suite: discovery, assertions, running**	Zero	2h	`tests/unit/test_products.py`, `tests/unit/test_checkout.py`, `tests/unit/test_drills.py`
P03-02	**Debug a failing test: tracebacks, breakpoints and pytest's debugging flags**	Zero	2h	`tests/unit/test_debug_lab.py`, `docs/debugging.md`
P03-03	**Parametrize and data-driven tests**	Zero	2h	`tests/unit/test_checkout_params.py`, `tests/unit/test_users_data.py`
P03-04	**Fixtures deep dive and conftest**	Hero	4h	`tests/conftest.py`, `tests/api/conftest.py`, `tests/api/booker/test_booking_fixture`
P03-05	**Markers, config and custom CLI options**	Hero	3h	`[tool.pytest.ini_options]` in `pyproject.toml`, `--env` option in `tests/conftest.py`
P03-06	**Hooks, plugins and coverage**	Super Hero	4h	`framework/pytest_plugin.py`, `tests/unit/test_plugin.py`

HOW THE RUN BUTTONS WORK HERE

Every test example runs **real pytest 8.1** in your browser. Choose pytest's options from the dropdown next to ▶ **Run pytest**, or press ✎ **Edit** to change the test and run it again. The first run downloads Python (about 10 MB). Your ticks in the **Done when** boxes are saved in this browser.

KEY **What the Run buttons run against** + ADDED

The browser has no network and no copy of your repo, so a few small files stand in for them:

FILE	WHAT IT IS
`python_basics/checkout.py, products.py, data_io.py`	P02's reference solutions — the code under test in P03's examples
`framework/data/saucedemo_users.csv`	the six SauceDemo users, as P02-03 asked
`framework/config.py`	a stand-in for your pydantic-settings file: same fields, defaults and QE_ prefix
`fakes/booker.py + a root conftest.py`	a simulated Restful-Booker. It plugs into the real requests stack, answers /ping with 201, needs the token for PUT/PATCH/DELETE (else 403), and refuses every other host

The simulated Restful-Booker, in full:

```
"""A simulated Restful-Booker for the lesson's Run buttons (browser sandbox and local
checks).

It plugs into the real `requests` stack at the transport layer, so code that uses
requests.Session, cookies, Basic auth, hooks and retry adapters runs unchanged.
Requests to the Restful-Booker hosts below are answered from memory; any other host
gets a ConnectionError, because the sandbox has no network.

It copies the behaviours the guides rely on: /ping answers 201, POST /auth returns a
token, PUT/PATCH/DELETE need the token cookie or Basic admin:password123 (otherwise
403), DELETE answers 201, and unknown ids answer 404. Invalid bodies get 400, or 500
when installed with buggy=True, to show what a "never 5xx" guard is for.
"""
from __future__ import annotations

import base64
import copy
import json
import secrets
from urllib.parse import parse_qs, urlsplit
```

```
import requests
from requests.adapters import HTTPAdapter
from requests.structures import CaseInsensitiveDict

HOSTS = {"restful-booker.herokuapp.com", "localhost:3001", "127.0.0.1:3001",
"restful-booker:3001"}
REASONS = {200: "OK", 201: "Created", 400: "Bad Request", 403: "Forbidden", 404: "Not
Found",
           405: "Method Not Allowed", 500: "Internal Server Error"}
FIELDS = {"firstname": str, "lastname": str, "totalprice": int, "depositpaid": bool,
"bookingdates": dict}
BASIC = "Basic " + base64.b64encode(b"admin:password123").decode()
MALFORMED = object()


class FakeBooker:
    def __init__(self) -> None:
        self.buggy = False
        self.reset()

    def reset(self) -> None:
        self.tokens: set[str] = set()
        self.bookings: dict[int, dict] = {}
        self.next_id = 1
        for first, last in (("Sally", "Brown"), ("Jim", "Wilson")):          # a
little seed data
            self._store({"firstname": first, "lastname": last, "totalprice": 111,
"depositpaid": True,
                         "bookingdates": {"checkin": "2026-01-01", "checkout": "2026-
01-02"},
                         "additionalneeds": "Breakfast"})

    def _store(self, booking: dict) -> int:
        booking_id, self.next_id = self.next_id, self.next_id + 1
        self.bookings[booking_id] = copy.deepcopy(booking)
        return booking_id

    @staticmethod
    def _json(raw: object) -> object:
        if raw is None:
            return None
        try:
            return json.loads(raw.decode() if isinstance(raw, bytes) else raw)
        except ValueError:
            return MALFORMED

    @staticmethod
    def _invalid(body: object) -> bool:
        if not isinstance(body, dict):
            return True
        for name, kind in FIELDS.items():
            value = body.get(name)
            if value is None or not isinstance(value, kind) or (kind is int and
isinstance(value, bool)):
                return True
        return not all(isinstance(body["bookingdates"].get(k), str) for k in
("checkin", "checkout"))

    def _bad(self) -> tuple[int, str]:
        return (500, "Internal Server Error") if self.buggy else (400, "Bad Request")
```

```
    def _authorised(self, headers: CaseInsensitiveDict) -> bool:
        if headers.get("Authorization") == BASIC:
            return True
        pairs = (part.strip().split("=", 1) for part in headers.get("Cookie",
"").split(";") if "=" in part)
        return dict(pairs).get("token") in self.tokens

    def handle(self, method: str, url: str, headers: CaseInsensitiveDict, raw:
object) -> tuple[int, object]:
        parts = urlsplit(url)
        path = parts.path.rstrip("/") or "/"
        query = {key: values[0] for key, values in parse_qs(parts.query).items()}
        segments = path.strip("/").split("/")
        if path == "/ping":
            return 201, "Created"
        if path == "/auth" and method == "POST":
            if self._json(raw) == {"username": "admin", "password": "password123"}:
                token = secrets.token_hex(8)
                self.tokens.add(token)
                return 200, {"token": token}
            return 200, {"reason": "Bad credentials"}
        if segments[0] != "booking" or len(segments) > 2:
            return 404, "Not Found"
        if len(segments) == 1:
            if method == "GET":
                wanted = {k: v for k, v in query.items() if k in ("firstname",
"lastname")}
                return 200, [{"bookingid": i} for i, b in self.bookings.items()
                             if all(b.get(k) == v for k, v in wanted.items())]
            if method != "POST":
                return 405, "Method Not Allowed"
            body = self._json(raw)
            if body is MALFORMED:
                return 400, "Bad Request"
            if self._invalid(body):
                return self._bad()
            return 200, {"bookingid": self._store(body), "booking": body}
        if not segments[1].isdigit():
            return 404, "Not Found"
        booking_id = int(segments[1])
        if method == "GET":
            return (200, self.bookings[booking_id]) if booking_id in self.bookings
else (404, "Not Found")
        if method not in ("PUT", "PATCH", "DELETE"):
            return 405, "Method Not Allowed"
        if not self._authorised(headers):
            return 403, "Forbidden"
        if booking_id not in self.bookings:
            return 404, "Not Found"
        if method == "DELETE":
            del self.bookings[booking_id]
            return 201, "Created"
        body = self._json(raw)
        if body is MALFORMED or not isinstance(body, dict):
            return 400, "Bad Request"
        if method == "PATCH":
            merged = copy.deepcopy(self.bookings[booking_id])
            for key, value in body.items():
                if isinstance(value, dict) and isinstance(merged.get(key), dict):
                    merged[key].update(value)
                else:
```

```
                    merged[key] = value
            body = merged
        if self._invalid(body):
            return self._bad()
        self.bookings[booking_id] = copy.deepcopy(body)
        return 200, body


SERVER = FakeBooker()


def _send(adapter: HTTPAdapter, request: requests.PreparedRequest, **kwargs: object)
-> requests.Response:
    host = urlsplit(request.url).netloc
    if host not in HOSTS:
        raise requests.ConnectionError(
            f"lesson sandbox: no network, so {host} is unreachable (only the
simulated Restful-Booker answers)",
            request=request)
    status, payload = SERVER.handle(request.method, request.url, request.headers,
request.body)
    is_json = not isinstance(payload, str)
    response = requests.Response()
    response.status_code, response.reason = status, REASONS.get(status, "")
    response._content = (json.dumps(payload) if is_json else payload).encode()
    response._content_consumed = True
    response.headers = CaseInsensitiveDict({
        "Content-Type": f"{'application/json' if is_json else 'text/plain'};
charset=utf-8",
        "X-Simulated-By": "lesson sandbox"})
    response.encoding, response.url, response.request, response.connection = "utf-8",
request.url, request, adapter
    return response


def install(buggy: bool = False) -> FakeBooker:
    """Route Restful-Booker hosts to the simulation for every requests session in
this process."""
    SERVER.reset()
    SERVER.buggy = buggy
    HTTPAdapter.send = _send
    return SERVER
```

PRIMERS — FUNDAMENTALS

📚 What happens when you type `pytest`

Three ideas carry this phase: pytest's **run lifecycle**, the shape of a **test function**, and **fixture scope**. Get these straight and every task below is a variation on them.

One run, five stages

pytest does the same five things every time. Most "why didn't my test run?" questions are answered by knowing which stage you're in.



EXIT CODE	MEANING	WHY CI CARES
0	all collected tests passed	the only green
1	some tests failed	a red build
2	interrupted (Ctrl-C, or errors during collection)	the run never finished
3	internal error	a plugin or pytest itself broke
4	usage error	bad option or config
5	no tests collected	**the fake-green trap**: a typo in `-m` runs nothing

⚠ SETUP VS CALL

A failure in a fixture is reported as an **error**; a failure in the test body is a **failure**. Errors usually mean environment or data trouble, not a product bug — that distinction speeds up triage.

The shape of a pytest test

A test is a plain function. Its **name** gets it collected, its **parameters** are fixtures pytest injects by name, its **decorators** are marks, and a plain **assert** is enough because pytest

rewrites it to show both sides on failure.

`def test_…():`

Collected because of the `test_` prefix, in a `test_*.py` file.

`(booking, tmp_path)`

Each parameter is a **fixture**, resolved by name — from `conftest.py`, a plugin, or pytest's built-ins.

`@pytest.mark.…`

Labels for selection (`-m smoke`) or behaviour (`skip` , `xfail` , `parametrize`).

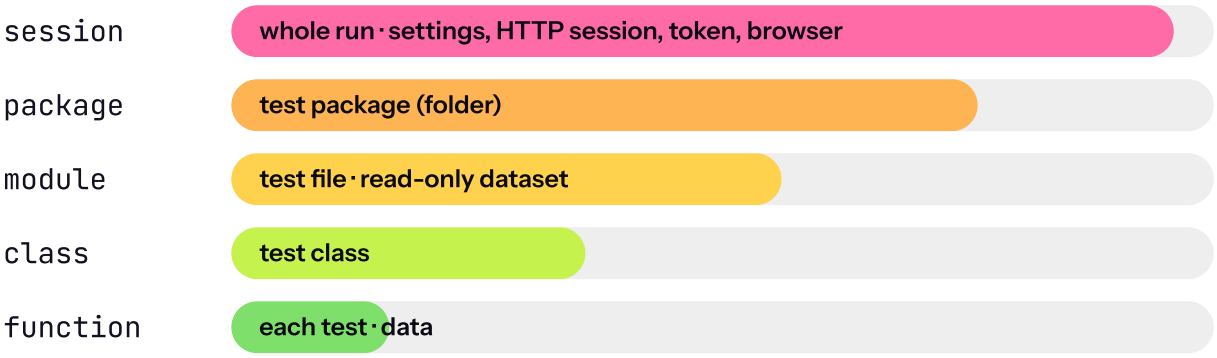
`assert a == b`

On failure pytest prints both values and the sub-expressions that produced them.

Fixture scope is how long a value lives

The wider the scope, the cheaper the run and the bigger the risk of shared state. Choose by **cost vs isolation**.

SCOPE LADDER — CREATED ONCE PER…



⚠ TWO RULES

1. A fixture may only use fixtures of the same or a **wider** scope (else `ScopeMismatch`). **2.** Teardown runs in **reverse** order of setup — last created, first destroyed.

🧰 Eight tools you'll reuse

Everything in P03 — and most of P04 and P05 — is built from these eight. Learn the tool, then the tasks are just where to apply it.

✅

Plain assert

pytest rewrites it to show both sides.

When: every check on a value you already have.

```
def test_tax_is_rounded_half_up():
    assert totals["tax"] ==
Decimal("2.40")
```

POWERS → **P03-01 · everywhere**

💥

pytest.raises

Expect an exception, and check its message.

When: invalid input must be rejected.

```
with
pytest.raises(CheckoutValidationError,
                    match="Last Name is
required"):
    validate_checkout("Ada", "",
"12345")
```

POWERS → **P03-01 · P03-03**

🧮

parametrize + ids

One body, many data rows, each reported on its own.

When: the same behaviour with different inputs.

```
@pytest.mark.parametrize("prices,
total", CASES,
                         ids=["one",
"two", "three"])
```

POWERS → **P03-03 · P04-03 negative matrix**

🧱

yield fixtures

Setup, hand over, teardown — teardown always runs.

When: a test needs a resource that must be cleaned up.

```
@pytest.fixture(scope="session")
def http():
    session = requests.Session()
    yield session          # tests run
here
    session.close()        # teardown,
always
```

POWERS → **P03-04 · P05 browsers**

Factory fixtures

Return a function; track everything it creates.

When: one test needs several resources with different data.

```
@pytest.fixture
def make_booking(http):
    created = []
    def _make(**overrides): ...
    yield _make
    for booking_id in
reversed(created): ...
```

POWERS → **P03-04 · P04-07**

Built-in fixtures

`tmp_path`, `monkeypatch`, `caplog`, `capsys`, `request`.

When: temp files, env overrides, log checks — all undone after the test.

```
def test_env(monkeypatch, tmp_path,
caplog):
    monkeypatch.setenv("QE_ENV",
"local")
```

POWERS → **P03-04**

Markers + -m

Label tests; select them with expressions.

When: CI gates run different subsets.

```
@pytest.mark.smoke
def test_ping(): ...
# pytest -m "smoke and not quarantine"
```

POWERS → **P03-05 · P09 gates**

Hooks

Functions pytest calls at fixed points of a run.

When: a rule should apply to every test without anyone remembering it.

```
def
pytest_collection_modifyitems(config,
items):
    for item in items: ...      # mark,
skip, reorder
```

POWERS → **P03-03 · P03-05 · P03-06**

P03-01 · ZERO · 2H

First pytest suite: discovery, assertions, running

App — —

Deliverable — `tests/unit/test_products.py`, `tests/unit/test_checkout.py`, `tests/unit/test_drills.py`

📋 The task

1. Move every P02 `assert` into pytest test functions (one behaviour per test).
2. Use `pytest.raises(..., match=...)` for exceptions and `pytest.approx` where floats are unavoidable.
3. Practise and note in `docs/pytest-cli.md` what each does: `-v`, `-q`, `-x`, `-k`, `--lf`, `--ff`, `-s`, `--tb=short`, `--durations=5`, `--co`.
4. Break one function on purpose and confirm a test fails with a readable message, then fix it.

done when Tick these off

- [ ] `uv run pytest tests/unit -q` is green
- [ ] Every test name reads like a sentence: `test_tax_is_rounded_half_up_to_cents`
- [ ] You've seen each of pytest's exit codes 0, 1 and 5

practise The flags from step 3, on one file + ADDED

Pick an option from the dropdown and press Run. Between them you'll see exit codes **0**, **1** and **5** — the third "done when" box.

tests/unit/test_cli_playground.py

```python
# tests/unit/test_cli_playground.py   (+ added: one file to practise pytest's flags on)
import time


def test_tax_is_eight_percent():
    assert round(29.99 * 0.08, 2) == 2.4


def test_slow_report():
    time.sleep(0.3)                      # pretend this one is slow: watch --durations
    assert True


def test_prints_something():
    print("you only see this line with -s")
    assert "Products".lower() == "products"


def test_broken_on_purpose():
    assert sorted([3, 1, 2]) == [3, 2, 1], "meant to fail: watch -x, -k and the exit code"
```



User guide

concepts What you need to know

- **Discovery:** files `test_*.py` or `*_test.py` → functions `test_*` → classes `Test*` (no `__init__`) with `test_*` methods.
- **Plain `assert`:** pytest *rewrites* asserts to show both sides on failure, so there's no `assertEqual` zoo.
- **AAA:** *Arrange* (set up), *Act* (do one thing), *Assert* (check the outcome).
- **Exit codes:** `0` all passed, `1` some failed, `2` interrupted, `3` internal error, `4` usage error, `5` no tests collected.

deep dive Going deeper

Assertion rewriting. At import time pytest rewrites `assert` statements in test modules so a failure prints intermediate values (`assert 2.39 == 2.40` plus where each came from). That's why helper modules containing asserts should be registered with `pytest.register_assert_rewrite`, or just keep asserts in tests.

Exit codes matter in CI. CI fails a job on non-zero exit. Exit code **5** (no tests collected) catches a real trap: a typo in `-m smok` silently runs *nothing*, and without treating 5 as failure your gate would be "green".

One behaviour per test. When a test fails its name should tell you what broke. Multiple asserts are fine if they check *one behaviour* (e.g. all three totals fields). Unrelated checks belong in separate tests.

Do your tests catch bugs? A test that never fails is decoration. **Mutation testing** (tools like `mutmut`) flips operators and constants in your code and checks that some test fails. Surviving mutants reveal weak assertions. Do the manual version now: break the code and watch the test go red.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: a first test and an exception test.

tests/unit/test_checkout.py

```
# tests/unit/test_checkout.py
from decimal import Decimal
import pytest
from python_basics.checkout import calculate_totals, validate_checkout,
CheckoutValidationError

def test_tax_is_eight_percent_rounded_to_cents():
    assert calculate_totals([Decimal("29.99")])["tax"] == Decimal("2.40")

def test_missing_last_name_is_rejected():
    with pytest.raises(CheckoutValidationError, match="Last Name is required"):
        validate_checkout("Ada", "", "12345")
```

SAY THIS

Each test name states the behaviour it protects, and `pytest.raises(..., match=...)` checks the exception type and its message in one line.

LEVEL 2 — IMPRESS THEM

🔵 Medium: grouping, AAA and approx.

tests/unit/test_sorting.py

```python
import pytest
from python_basics.products import PRODUCTS, sort_products, is_sorted

class TestSorting:
    def test_low_to_high_orders_prices_ascending(self):
        # Arrange
        products = list(PRODUCTS)
        # Act
        result = sort_products(products, "lohi")
        # Assert
        assert is_sorted([p["price"] for p in result])

    def test_unknown_mode_fails_fast(self):
        with pytest.raises(KeyError):
            sort_products(PRODUCTS, "random")

def test_average_rating_float():
    assert (0.1 + 0.2) == pytest.approx(0.3)      # floats: approx, money: Decimal
```

HIRE SIGNAL

The class is only organisation; AAA keeps each test to one action, and `approx` is for floats — money stays Decimal.

LEVEL 3 — HERO MODE

🟣 High/Complex: prove tests catch bugs (manual mutation testing).

```
# 1. Mutate: in checkout.py change ROUND_HALF_UP → ROUND_DOWN
uv run pytest tests/unit/test_checkout.py -q     # must FAIL; if green, your tests are too weak
# 2. Mutate: change `if not value.strip()` → `if not value`
uv run pytest -k "whitespace" -q                  # do you have a test for "   " input? add one
# 3. Restore and run with details
uv run pytest -q --durations=5 --tb=short
# Automated version (explore): uv add --dev mutmut, then follow its docs to run on python_basics/
```

Write in `docs/pytest-cli.md` which mutants survived and the test you added to kill each.

WHY IT'S HERO-LEVEL

Mutation testing answers the question coverage can't: if I break the code on purpose, does a test go red?

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **How does pytest find tests?**

From the rootdir or `testpaths`, it collects files named `test_*.py` or `*_test.py`, then functions starting with `test`, and classes starting with `Test` (without an `__init__`) containing `test*` methods. You can customise this with `python_files`, `python_classes` and `python_functions` in config. `pytest --co` shows what would be collected.

H **Why can pytest use plain `assert` when unittest needs `assertEqual`?**

pytest rewrites the assert statements' AST at import time to capture sub-expression values, so failures show detailed diffs (including for dicts and lists). unittest predates this and uses methods to produce messages.

H **Your CI job ran `pytest -m smok` and was green. What happened?**

The typo matched no tests, so pytest exited with code 5 ("no tests collected"). If CI treats that as success (or the step ignores it), the gate is fake-green. Prevent it with registered markers plus `--strict-markers` (which catches unknown markers used *on tests*), by failing on exit code 5, and by checking collected-test counts in the job summary.

Z **What is the AAA pattern?**

Arrange, Act, Assert: set up preconditions, perform the single action under test, then verify the outcome. It keeps tests readable and focused on one behaviour. (BDD's Given/When/Then is the same idea.)

SH **How do you know your test suite actually catches bugs?**

Coverage only says code *ran*, not that it was *checked*. Mutation testing measures the suite's ability to detect injected faults. Also: review assertions for strength (exact values, not just "not None"), check historical defect escape rate (bugs found in prod that tests should have caught), and deliberately break features in a sandbox (the Toolshop with-bugs version is great for this).

🤖 AI ASSIST

Ask AI to generate *mutants* of your function (small code changes) and check whether your tests kill each one.

P03-02 · 🟢 ZERO · 2H

Debug a failing test: tracebacks, breakpoints and pytest's debugging flags

App — — (a debugging lab) **Deliverable** — `tests/unit/test_debug_lab.py` , `docs/debugging.md`

📋 The task

1. Create `tests/unit/test_debug_lab.py` from the 🟢 example and run `uv run pytest tests/unit/test_debug_lab.py` . Three tests fail, each for a different reason.
2. For each failure, **read the output bottom-up and write a hypothesis *before* changing anything.**
3. Use every tool at least once: `-x` , `--lf` , `-l` (`--showlocals`), `--pdb` (then `p` , `pp` , `l` , `w` , `u` , `q`), a `breakpoint()` in the code, and your **IDE's debugger** with a breakpoint (VS Code *Debug Test*, or PyCharm's bug icon).
4. Run the failing cart test **on its own**. It passes. Explain why before you fix it.
5. Fix the *code under test*, not the assertions. (If you ever decide a test is wrong, say why in your notes.)

6. In `docs/debugging.md`, for each bug record the symptom, your hypothesis, the evidence and the tool that gave it, the root cause, the fix, and how you'd prevent it next time.

`done when` Tick these off

- [ ] All lab tests pass, both together and one at a time
- [ ] `docs/debugging.md` has symptom → hypothesis → evidence → root cause → fix → prevention for all three bugs
- [ ] You've used `--pdb` and an IDE breakpoint at least once each, and can say when you'd reach for each

📘 User guide

`concepts` What you need to know

- **Traceback:** the chain of calls that led to an error. Read it **bottom-up**: the last line is the error, and the frames above show how you got there. Find the first frame in *your* code.
- **Assertion introspection:** pytest rewrites `assert` so a failure shows the values on both sides (`Decimal('1') == Decimal('1299.00')`). Read those values before anything else.
- **Useful flags:** `-x` (stop at the first failure), `--lf` (re-run only last failures), `-l` (show local variables in tracebacks), `-s` (show `print` output), `-k expr` (select by name), `-vv` (full diffs), `--pdb` (open the debugger at the failure), `--trace` (open it at the start of each test).
- **pdb:** Python's built-in debugger. `p expr` prints, `pp` pretty-prints, `l` lists code, `w` shows where you are, `u` / `d` move up and down the stack, `n` runs the next line, `s` steps into a call, `c` continues, `q` quits.
- **`breakpoint()`:** a line you put in code to pause there and open the debugger (pytest disables output capture automatically when it hits one).
- **IDE debugger:** the same thing with a UI: click in the margin to set a breakpoint, run the test in debug mode, and inspect variables with the mouse.

`deep dive` Going deeper

Debugging is the scientific method, done quickly. Observe the symptom, form a hypothesis ("the price parser drops everything after the comma"), predict what you'd see if it were true, and run the cheapest experiment that could prove it wrong: print a value, run one

test alone, or pause in the debugger. Changing code at random until the test goes green isn't debugging; it's how new bugs get in.

Each lab bug teaches a pattern you'll meet at work:

- **Wrong value:** the assertion shows the values. Ask *which input* produces it, and whether that input is a boundary (a thousands separator is a boundary that the "happy" test didn't include).
- **Passes alone, fails with others:** shared state between tests. Here it's a Python classic, a **mutable default argument** (`cart=[]` is created once and shared by every call). In real suites the shared state is a database row, a logged-in browser or a global variable (P05-04, P07-02). Note the `# noqa: B006` in the lab: ruff *would* have caught this bug, and someone had switched that check off.
- **An exception deep in a data structure:** `KeyError: 'checkOut'` means the key isn't there. `--pdb` lets you type `p dates.keys()` and see instantly that the API uses `checkin` / `checkout`. Your code assumed a different contract (P04-08 is all about contracts).

print vs logging vs the debugger: `print` is fine for a 30-second experiment (run with `-s`). Use `logging` for anything you'd want to see in CI later, because pytest captures it and shows it only for failing tests (`--log-cli-level=INFO` shows it live). Reach for the debugger when you don't yet know *which* value to look at: pause, look around, move up the stack.

When the bug is "it worked yesterday": narrow it down. What changed (code, data, dependency versions, environment)? `git bisect` can find the exact commit that broke a test automatically (🟣 example). Dependency changes show up in the `uv.lock` diff.

> IN THIS PAGE
>
> The lab below really runs. `--pdb` and `breakpoint()` need a keyboard and a terminal, so try those in your repo; here, use the dropdown to compare `-x -l`, `-k new_cart` (it **passes** on its own) and `-vv` on the same three bugs.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 **Low: the debugging lab.** (copy it exactly, bugs included)

tests/unit/test_debug_lab.py

```python
# tests/unit/test_debug_lab.py: three bugs, three debugging tools. Fix the CODE, not the asserts.
from datetime import date
from decimal import Decimal


def parse_price(text: str) -> Decimal:
    """'$29.99' -> Decimal('29.99') and '$1,299.00' -> Decimal('1299.00')."""
    return Decimal(text.lstrip("$").split(",")[0])


def add_to_cart(item: str, cart: list[str] = []) -> list[str]:  # noqa: B006
    cart.append(item)
    return cart


def nights(booking: dict) -> int:
    dates = booking["bookingdates"]
    return (date.fromisoformat(dates["checkOut"]) -
date.fromisoformat(dates["checkIn"])).days


BOOKING = {"firstname": "Jim", "bookingdates": {"checkin": "2026-01-01", "checkout": "2026-01-04"}}


def test_parse_simple_price() -> None:
    assert parse_price("$29.99") == Decimal("29.99")


def test_parse_price_with_thousands_separator() -> None:
    assert parse_price("$1,299.00") == Decimal("1299.00")


def test_first_item_goes_into_an_empty_cart() -> None:
    assert add_to_cart("backpack") == ["backpack"]


def test_every_new_cart_starts_empty() -> None:
    assert add_to_cart("bike light") == ["bike light"]


def test_nights_for_a_three_night_booking() -> None:
    assert nights(BOOKING) == 3
```

```
uv run pytest tests/unit/test_debug_lab.py                    # 3 failures
uv run pytest tests/unit/test_debug_lab.py -x -l              # stop at the first, show locals
uv run pytest tests/unit/test_debug_lab.py -k new_cart        # the cart test alone: it passes!
uv run pytest tests/unit/test_debug_lab.py --lf --pdb         # re-run the failures, debug each
```

SAY THIS

I read each failure bottom-up and write a hypothesis before I change anything: three failures, three different causes.

LEVEL 2 — IMPRESS THEM

🔵 Medium: a post-mortem pdb session on the `KeyError`.

```
$ uv run pytest tests/unit/test_debug_lab.py -k nights --pdb
E       KeyError: 'checkOut'
>>> entering PDB >>>
> tests/unit/test_debug_lab.py(18)nights()
→ return (date.fromisoformat(dates["checkOut"]) -
date.fromisoformat(dates["checkIn"])).days
(Pdb) p dates.keys()
dict_keys(['checkin', 'checkout'])
(Pdb) u
> tests/unit/test_debug_lab.py(41)test_nights_for_a_three_night_booking()
→ assert nights(BOOKING) == 3
(Pdb) pp BOOKING
{'bookingdates': {'checkin': '2026-01-01', 'checkout': '2026-01-04'},
 'firstname': 'Jim'}
(Pdb) q
```

Two commands answered the question: the data uses lowercase keys, so the code's assumption is wrong. The fix belongs in `nights()`, and the *prevention* is a typed model that fails loudly on the wrong shape (pydantic, P04-04).

HIRE SIGNAL

Two pdb commands turned a guess into evidence. The fix goes in `nights()`; the prevention is a typed model that fails loudly on the wrong shape.

LEVEL 3 — HERO MODE

🟣 High/Complex: debug what you can't reproduce locally.

When an API test fails only in CI, make every HTTP exchange visible in the failure report:

tests/api/conftest.py

```
# tests/api/conftest.py (you'll build this folder in P04)
import logging

import pytest
import requests

log = logging.getLogger("http")


def _log_exchange(response: requests.Response, *args: object, **kwargs: object) -> None:
    request = response.request
    elapsed_ms = response.elapsed.total_seconds() * 1000
    log.info("%s %s → %s in %.0f ms", request.method, request.url, response.status_code,
elapsed_ms)
    log.debug("request body: %s", request.body)
    log.debug("response body: %s", response.text[:500])


@pytest.fixture
def http() → requests.Session:
    session = requests.Session()
    session.hooks["response"].append(_log_exchange)
    return session
```

pytest attaches captured logs to each *failing* test's report, so a CI failure now shows the exact requests and responses. Run with `--log-cli-level=INFO` to watch them live.

When a test that used to pass starts failing, let Git find the guilty commit:

```
git bisect start
git bisect bad                    # the current commit fails
git bisect good v0.3.0            # this tag passed
git bisect run uv run pytest tests/unit/test_pricing.py -q
git bisect reset
```

`git bisect run` binary-searches the history, running the test at each step, and names the first bad commit, usually in a handful of steps even across hundreds of commits.

> WHY IT'S HERO-LEVEL
>
> Make CI failures explain themselves: log every HTTP exchange so the report carries the evidence, and let `git bisect run` find the commit.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **How do you read a Python traceback?**

Bottom-up. The last line is the exception type and message; the frames above show the call chain that led there, most recent last. Find the first frame in *your* code rather than the library's, read the line and the local values (`-l` helps), and form a hypothesis about why the data or state was wrong at that point.

Z **A test passes on its own but fails when run with the others. What's likely going on?**

Shared state: one test leaves something behind that another depends on or is broken by, such as a global or class variable, a mutable default argument, a database row, a logged-in browser session or a file. Prove it by running the pair in different orders, then fix it by giving each test its own fresh state through fixtures, unique data and teardown.

Z **Which pytest options do you use most when debugging?**

`-x` to stop at the first failure, `--lf` to re-run only the last failures, `-k` to select one test, `-l` to show local variables, `-vv` for full diffs, `-s` to see prints, `--log-cli-level` to see logs live, and `--pdb` to open the debugger where it failed.

H **When do you use a debugger rather than print statements or logs?**

Print is fine for a quick check of one value you already suspect. Logging is for information you'll want in CI and on future failures. A debugger is best when you don't yet know what to look at: you can pause at the failure, inspect any variable, walk up the call stack and try expressions interactively, without re-running the test for each guess.

H **A test fails only in CI. How do you debug it?**

Get the evidence CI already has: logs, the JUnit report, traces, screenshots, captured HTTP exchanges. Compare environments (versions, config, secrets, data, time zone, parallelism, resources). Reproduce CI's conditions locally: the same container image, the same flags (`-n`, random order), the same env vars. Add the missing observability for next time, and avoid "fixing" it with retries until you understand it.

SH **How do you help a team get faster at diagnosing test failures?**

Make failures self-explanatory: assertion messages with context, captured HTTP exchanges and traces attached to every failure, and a failure summary in the CI job page. Classify failures (product bug, test bug, environment, flaky) and track the time from red build to diagnosis. Keep a runbook of common failure signatures, and automate the first triage step (P12-04's triage agent does exactly that).

🤖 AI ASSIST

- Paste **only** the failure output (no fixes yet) and ask AI for three competing hypotheses and the cheapest experiment to tell them apart. Then run the experiments yourself.
- Don't ask for "the fix" until you've written your own hypothesis in `docs/debugging.md`. That hypothesis is the skill interviewers test in live-coding rounds.

P03-03 · 🟢 ZERO · 2H

Parametrize and data-driven tests

App — SauceDemo data

Deliverable — `tests/unit/test_checkout_params.py`, `tests/unit/test_users_data.py`

📋 The task

1. Parametrize `calculate_totals` for 1, 2 and 3 items with readable `ids`.
2. Parametrize `validate_checkout` for each missing field, including whitespace-only values.
3. Load `saucedemo_users.csv` (P02-03) into a parametrize decorator with `ids=username`.
4. Use `pytest.param(..., marks=pytest.mark.xfail(reason="..."))` for one known-bug case.
5. Stack two parametrize decorators and observe the cartesian product in `pytest --co -q`.

`done when` Tick these off

- [ ] `pytest --co -q` shows readable IDs like `test_login_data[locked_out_user]`
- [ ] Every parametrized case is reported separately

📘 User guide

`concepts` What you need to know

- `@pytest.mark.parametrize("a,b", [(1, 2), (3, 4)])`: run one test body per data row.
- `ids`: names for each case in reports (a list or a function).
- `pytest.param(values, id=..., marks=...)`: a per-case id or marks (xfail/skip).
- **Stacking:** two parametrize decorators give every combination (cartesian product).

`deep dive` Going deeper

Parametrize vs a loop inside a test. A loop stops at the first failure and reports one test. Parametrize runs *every* case independently, reports each separately, and supports `--lf` re-runs of just the failed case. Always parametrize.

Parametrize the *data*, not the *behaviour*. "Valid login" and "locked-out login" check different outcomes. You can parametrize them with an `expected` column, but if the test body needs `if expected_error:` branches everywhere, split it into two tests. Branches inside tests are a smell.

Combinatorial explosion. 3 browsers × 6 users × 4 sort modes = 72 runs. Stack only what truly interacts. For large input spaces use **pairwise testing** (every *pair* of values covered at least once), which cuts combinations dramatically with little loss in defect detection.

Collection-time data. Parametrize arguments are evaluated when the module is imported, so a broken CSV fails *collection* for the whole file with a clear error. That's good: bad data is caught before anything runs.

INTERACTIVE — THE SAME THREE CASES, TWO WAYS

```
test_totals · case one-item ✓
test_totals · case two-items ✗ AssertionError
case three-items never checked
1 test · 1 failed
```

The report shows one failing test. `--lf` would re-run all three cases again.

task step 5 Stacked parametrize + ADDED

Two decorators, 3 × 2 = 6 tests. Run with `--co -q` to see the ids pytest builds.

tests/unit/test_stacked.py

```python
# tests/unit/test_stacked.py   (+ added: task step 5, the cartesian product)
import pytest


@pytest.mark.parametrize("browser", ["chromium", "firefox"])
@pytest.mark.parametrize("user", ["standard_user", "problem_user", "visual_user"])
def test_login_matrix(browser, user):
    assert browser and user
```

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 **Low: parametrize with ids.**

tests/unit/test_checkout_params.py

```python
import pytest
from decimal import Decimal
from python_basics.checkout import calculate_totals

@pytest.mark.parametrize(
    "prices, expected_total",
    [
        ([Decimal("29.99")], Decimal("32.39")),
        ([Decimal("29.99"), Decimal("9.99")], Decimal("43.18")),
        ([Decimal("7.99"), Decimal("15.99"), Decimal("49.99")], Decimal("79.89")),
    ],
    ids=["one-item", "two-items", "three-items"],
)
def test_total_includes_tax(prices, expected_total):
    assert calculate_totals(prices)["total"] == expected_total
```

NOTE

Check the expected totals against the SauceDemo UI yourself. That *is* the exercise.

SAY THIS

One body, three data rows, three independently reported tests — with ids that read well in the report.

LEVEL 2 — IMPRESS THEM

🔵 **Medium: data from CSV plus a known-bug xfail.**

tests/unit/test_users_data.py

```python
import pytest
from python_basics.data_io import load_csv

USERS = load_csv("saucedemo_users.csv")

@pytest.mark.parametrize("user", USERS, ids=lambda u: u["username"])
def test_user_record_is_consistent(user):
    # a user that cannot log in must have an expected error message
    assert user["can_login"] or user["expected_error"]

@pytest.mark.parametrize("first, last, postal, message", [
    ("", "Doe", "123", "First Name is required"),
    ("Ada", "", "123", "Last Name is required"),
    ("Ada", "Doe", "", "Postal Code is required"),
    pytest.param("   ", "Doe", "123", "First Name is required", id="whitespace-first"),
    pytest.param("Ada", "Doe", "ABC", "Postal Code is invalid", id="letters-postal",
                 marks=pytest.mark.xfail(reason="BUG-12: validator doesn't check postal
format yet", strict=True)),
])
def test_checkout_validation(first, last, postal, message):
    from python_basics.checkout import validate_checkout, CheckoutValidationError
    with pytest.raises(CheckoutValidationError, match=message):
        validate_checkout(first, last, postal)
```

HIRE SIGNAL

The rows come from the CSV at collection time, the whitespace case has its own id, and the known bug is a strict xfail that will shout when it's fixed.

LEVEL 3 — HERO MODE

🟣 High/Complex: dynamic parametrization with the `pytest_generate_tests` hook.

tests/unit/conftest.py

```python
# tests/unit/conftest.py
from python_basics.data_io import load_csv

def pytest_generate_tests(metafunc):
    """Any test that asks for `sauce_user` is run once per CSV row; `--only-login-ok`
filters rows."""
    if "sauce_user" in metafunc.fixturenames:
        rows = load_csv("saucedemo_users.csv")
        if metafunc.config.getoption("--only-login-ok", default=False):
            rows = [r for r in rows if r["can_login"]]
        metafunc.parametrize("sauce_user", rows, ids=[r["username"] for r in rows])

def pytest_addoption(parser):
    parser.addoption("--only-login-ok", action="store_true", help="only users that can log
in")
```

tests/unit/test_dynamic.py

```python
# tests/unit/test_dynamic.py
def test_username_has_no_spaces(sauce_user):
    assert " " not in sauce_user["username"]
```

The hook decouples *which data* from *which test*. The same pattern powers "run every test against every environment/browser" plugins.

WHY IT'S HERO-LEVEL

`pytest_generate_tests` separates *which data* from *which test* — the pattern behind every run-per-environment plugin.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **Why parametrize instead of looping inside one test?**

Each case becomes its own test: independent pass/fail, separate report lines, re-runnable with `--lf`, and one failure doesn't hide the others. A loop stops at the first failing assert and reports a single result.

H **How do you drive tests from CSV or Excel?**

Load the file at collection time (csv module, openpyxl or pandas for Excel), convert types explicitly, and feed rows into `parametrize` with ids from a meaningful column. Validate the data (required columns) so a bad row fails collection clearly. For large or shared datasets, use the `pytest_generate_tests` hook so tests declare what data they need rather than where it comes from.

H **What happens when you stack two parametrize decorators?**

You get the cartesian product: every value of the first combined with every value of the second. Two lists of 3 give 9 tests. It's useful for real interactions (browser × viewport) but explodes quickly, so use it deliberately.

SH **Your data-driven suite has 5 parameters with 4 values each (1,024 combos). What do you do?**

Don't run all of them. Use equivalence partitioning to cut values per parameter, then pairwise (all-pairs) generation to cover every pair of values in roughly 16–25 cases. Add targeted cases for known risky combinations and boundary values. Run the exhaustive set nightly only if it's cheap (API level), never in PR gates.

SH **When would you use `pytest_generate_tests` instead of the decorator?**

When parameters depend on runtime config (CLI options, environment, a file chosen at run time), when many tests share the same dataset by fixture name, or when building a plugin (e.g. running every test per tenant). The decorator is static; the hook is dynamic.

🤖 AI ASSIST

Give AI your input fields and ask for an **equivalence-partition + boundary table**, then turn the good rows into `pytest.param` cases.

P03-04 · 🔵 HERO · 4H

Fixtures deep dive and conftest

App — Restful-Booker

Deliverable — `tests/conftest.py`, `tests/api/conftest.py`, `tests/api/booker/test_booking_fixtures.py`

📋 The task

1. `tests/conftest.py`: a **session** fixture `settings` (from P02-05) and a session fixture `http` (a `requests.Session` closed at teardown).
2. `tests/api/conftest.py`: a session fixture `booker_token` (`POST /auth`) and a **function** fixture `booking` that creates a booking and deletes it after the test (yield).

3. A **factory fixture** `make_booking(**overrides)` that can be called many times in one test and deletes *everything* it created at teardown.
4. Use built-ins: `tmp_path` (write a file), `monkeypatch` (override an env var), `caplog` (assert a log line), `request` (read the test name).
5. Run `pytest --setup-show` and explain the output in `docs/fixtures.md`.

`done when` Tick these off

- [ ] No booking created by your tests remains after the run, even when a test fails
- [ ] `--setup-show` shows the token fetched once per session and bookings per test

📘 User guide

`concepts` What you need to know

- **Fixture:** a function that provides a resource to tests that name it as a parameter (dependency injection).
- **Scope:** `function` (default), `class`, `module`, `package`, `session`. This is how long a fixture's value is cached.
- `yield` **fixture:** code before `yield` is setup, after it is teardown (it runs even if the test fails).
- **Factory fixture:** returns a *function* so tests can create N resources with different data.
- `conftest.py`**:** fixtures here are visible to all tests in that directory and below, with no import needed.
- **Built-ins:** `tmp_path`, `monkeypatch`, `caplog`, `capsys`, `request`, `pytestconfig`.

`deep dive` Going deeper

How pytest resolves fixtures. For each test, pytest reads parameter names, finds fixtures with those names (nearest `conftest.py` wins, which lets you **override** a fixture in a subfolder), builds the dependency graph, and executes it: higher scopes first, cached per scope. Teardown runs in **reverse (LIFO)** order.

Scope rules. A fixture can depend only on fixtures of the **same or wider** scope. A session `booker_token` can use a session `http`, but a session fixture requesting a function fixture raises `ScopeMismatch`. Choose scope by *cost vs isolation*:

RESOURCE	SCOPE	WHY
Settings, HTTP session, auth token	session	Expensive, read-only
Browser (pytest-playwright)	session	Expensive to launch
Browser context/page	function	Isolation: fresh cookies/storage
Created test data	function	Each test owns its data

Factory fixtures solve "I need 3 bookings with different names" without writing 3 fixtures, and centralise cleanup: the factory records every ID it creates and deletes them all in teardown.

Avoid sharing state between tests. Tests that pass data through globals or depend on order break under `--lf`, `-k`, random order and parallel runs (P07). Share **setup** via fixtures, never **results** via variables.

autouse=True runs a fixture for every test in scope without being requested. It's handy for global concerns (logging context, screenshot on failure) but invisible magic otherwise, so use it sparingly.

INTERACTIVE — WHAT --SETUP-SHOW WOULD PRINT

```
SETUP S http session: created once
SETUP S booker_token needs http
SETUP F booking test_a's own booking
test_a PASSED
TEARDOWN F booking DELETE /booking/{id}
SETUP F booking test_b's own booking
test_b PASSED
TEARDOWN F booking DELETE /booking/{id}
TEARDOWN S booker_token
TEARDOWN S http session.close()
```

…so http, created first, closes last. Teardown is LIFO.



Worked examples

LEVEL 1 — CRACK IT

🟢 **Low: session fixtures with teardown.**

tests/conftest.py

```python
# tests/conftest.py
import pytest, requests
from framework.config import get_settings

@pytest.fixture(scope="session")
def settings():
    return get_settings()

@pytest.fixture(scope="session")
def http():
    session = requests.Session()
    session.headers.update({"Accept": "application/json"})
    yield session                 # ← tests run here
    session.close()               # teardown, always
```

SAY THIS

Session scope because the HTTP session is expensive and read-only; `yield` so the teardown always runs.

LEVEL 2 — IMPRESS THEM

🔵 Medium: auth token plus a create/delete data fixture.

tests/api/conftest.py

```python
# tests/api/conftest.py
import pytest

@pytest.fixture(scope="session")
def booker_url(settings):
    return settings.booker_url

@pytest.fixture(scope="session")
def booker_token(http, booker_url, settings):
    r = http.post(f"{booker_url}/auth", json={"username": "admin", "password":
"password123"},
                  timeout=settings.http_timeout)
    r.raise_for_status()
    return r.json()["token"]

@pytest.fixture
def booking(http, booker_url, booker_token, settings):
    from python_basics.checkout import build_booking   # P02-02 builder
    r = http.post(f"{booker_url}/booking", json=build_booking(),
timeout=settings.http_timeout)
    r.raise_for_status()
    created = r.json()                                  # {"bookingid": 123, "booking":
{...}}
    yield created
    http.delete(f"{booker_url}/booking/{created['bookingid']}",
                cookies={"token": booker_token}, timeout=settings.http_timeout)
```

HIRE SIGNAL

The token is fetched once per session, while each booking is created and deleted per test, so every test owns its data.

LEVEL 3 — HERO MODE

🟣 High/Complex: a factory fixture with tracked cleanup, built-ins and overrides.

tests/api/conftest.py

```
# tests/api/conftest.py (continued)
import logging
import pytest

log = logging.getLogger("fixtures")

@pytest.fixture
def make_booking(http, booker_url, booker_token, settings, request):
    created: list[int] = []

    def _make(**overrides):
        from python_basics.checkout import build_booking
        r = http.post(f"{booker_url}/booking", json=build_booking(**overrides),
timeout=settings.http_timeout)
        r.raise_for_status()
        booking_id = r.json()["bookingid"]
        created.append(booking_id)
        log.info("[%s] created booking %s", request.node.name, booking_id)
        return r.json()

    yield _make

    for booking_id in reversed(created):                # LIFO cleanup
        resp = http.delete(f"{booker_url}/booking/{booking_id}",
                           cookies={"token": booker_token}, timeout=settings.http_timeout)
        if resp.status_code ⩾ 400:
            log.warning("cleanup failed for %s: %s", booking_id, resp.status_code)
```

tests/api/booker/test_booking_fixtures.py

```python
# tests/api/booker/test_booking_fixtures.py
import logging

def test_filter_by_lastname_returns_only_matches(make_booking, http, booker_url, caplog):
    caplog.set_level(logging.INFO)
    a = make_booking(lastname="Zyx-Unique-1")
    make_booking(lastname="Other")
    ids = [b["bookingid"] for b in http.get(f"{booker_url}/booking",
                                            params={"lastname": "Zyx-Unique-1"},
timeout=10).json()]
    assert a["bookingid"] in ids
    assert "created booking" in caplog.text

def test_env_override(monkeypatch):
    monkeypatch.setenv("QE_BOOKER_URL", "http://localhost:3001")   # auto-restored after
test
    from framework.config import Settings
    assert Settings().booker_url == "http://localhost:3001"

def test_writes_report(tmp_path):
    report = tmp_path / "report.txt"                               # unique temp dir per
test
    report.write_text("ok")
    assert report.read_text() == "ok"
```

Runs against a simulated Restful-Booker inside your browser (see the phase map). Run it against the real API in your repo.

Run `uv run pytest tests/api/booker -q --setup-show` and read the `SETUP S` / `SETUP F` lines.

> WHY IT'S HERO-LEVEL
>
> The factory records each ID the moment it exists and deletes them in reverse order, so even a failing test leaves nothing behind.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **What are fixture scopes? Give an example of each.**

`function` (default; created per test, e.g. test data or a browser page), `class` (shared by a test class), `module` (per file, e.g. a read-only dataset), `package` (per directory package), `session` (once per run, e.g. settings, HTTP session, browser, auth token). Wider scope is faster but risks shared state.

Z **What is `conftest.py`?**

A special file where fixtures and hooks are defined for its directory and all subdirectories. Tests use them by name without importing. Nested conftests can add or override fixtures for a subtree.

H **`yield` fixture vs `request.addfinalizer`?**

Both register teardown. `yield` is simpler and reads top to bottom. `addfinalizer` lets you register multiple finalizers dynamically, and registers teardown *before* risky setup completes (if setup fails after the `yield`-less part, `yield` teardown won't run). Modern code mostly uses `yield` and splits risky setup into separate fixtures.

H **What is a ScopeMismatch error?**

A fixture requested a fixture with a *narrower* scope, e.g. a session-scoped fixture depending on a function-scoped one. The wider fixture would outlive its dependency. Fix by widening the dependency's scope or narrowing the requester's.

H **How do you share data between tests?**

You shouldn't share *results* between tests: each test must be independent and order-agnostic (so it survives `-k`, `--lf`, random order and parallel runs). Share *setup* through fixtures at the appropriate scope, and create per-test data via factory fixtures.

SH **How does fixture teardown behave when a test fails, or when setup fails halfway?**

If the test fails, all yield fixtures still resume and run teardown in reverse order. If a fixture's setup raises *before* its `yield`, its own teardown doesn't run (it never yielded), but already-created fixtures still tear down. So make each fixture own exactly one resource, making partial failure clean. Factory fixtures should record IDs immediately after creation.

🤖 AI ASSIST

Paste your `--setup-show` output into AI and ask it to explain the order, then check the explanation against the pytest docs on fixture scopes.

P03-05 · 🔵 HERO · 3H

Markers, config and custom CLI options

App — all

Deliverable — `[tool.pytest.ini_options]` in `pyproject.toml`, `--env` option in `tests/conftest.py`

📋 The task

1. Extend the `[tool.pytest.ini_options]` section the template gave you (P00-04): register the markers `smoke`, `regression`, `api`, `ui`, `db`, `slow` and `quarantine`. `--strict-markers` is already on; prove it by running a test with a misspelled marker.
2. Mark your existing tests. Practise `-m "smoke and not ui"`.
3. Add `--env {public,local,docker}` that switches all URLs (public demo apps / localhost / Docker service names).
4. `skipif` DB tests when `--env public` (no DB access). `xfail(strict=True)` a known bug with a link.
5. Configure `log_cli` and a log format in the pytest config.

done when Tick these off

- [ ] `pytest --env local -m smoke` targets localhost URLs (prove it with a log line)
- [ ] An unregistered marker causes an error

- [ ] `docs/markers.md` explains which markers each CI gate runs

📘 User guide

concepts What you need to know

- **Marker:** a label on a test (`@pytest.mark.smoke`) used for selection (`-m`) or behaviour (`skip` , `xfail`).
- `--strict-markers` : unknown markers become errors, so typos can't hide tests.
- `skip` / `skipif` : don't run (not applicable here). `xfail` : run it, and a failure is *expected* (a known bug). `strict=True` fails when it unexpectedly passes.
- `pytest_addoption` + `request.config.getoption()` : your own CLI flags.
- **ini options:** `addopts` , `testpaths` , `markers` , `log_cli` , `log_cli_level` , `xfail_strict` .

deep dive Going deeper

Markers are your gate API. CI gates select tests by marker: G3 runs `-m "smoke and not quarantine"` , G4 runs `-m "not quarantine"` . That's why marker names must be a controlled vocabulary (`--strict-markers`) and documented.

skip vs xfail semantics:

	RUNS?	REPORTED AS	USE FOR
`skip` / `skipif`	No	skipped	Not applicable (wrong env or OS, feature off)
`xfail`	Yes	xfailed / xpassed	Known bug, tracked
`xfail(strict=True)`	Yes	**fail** if it passes	Known bug: alerts you when it's fixed so you remove the mark

Environment selection. Prefer *one* switch (`--env`) that maps to a config profile over many URL flags. Precedence: CLI flag > env var > `.env` > defaults. Note that `pytest-playwright` (via `pytest-base-url`) already defines `--base-url` , so don't clash with it.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: register and use markers.

pyproject.toml

```
# pyproject.toml
[tool.pytest.ini_options]
addopts = "-ra --strict-markers"
testpaths = ["tests"]
pythonpath = ["."]             # keep this from the template: tests import framework/ and
python_basics/
markers = [
  "smoke: fast critical-path checks (PR gate)",
  "regression: full coverage (main/nightly)",
  "api: API layer", "ui: browser tests", "db: needs database access", "e2e: crosses
API/UI/DB",
  "slow: > 10s", "quarantine(issue): known flaky, excluded from gates",
  "serial: must not run concurrently (P07)", "live: calls a real LLM API (P12)",
]
log_cli = false
log_cli_level = "INFO"
log_format = "%(asctime)s %(levelname)s %(name)s: %(message)s"
```

tests/api/test_markers.py

```
import pytest

@pytest.mark.smoke
@pytest.mark.api
def test_ping(http, booker_url): ...
```

SAY THIS

Every marker is registered with a description, and `--strict-markers` turns a typo on a test into an error instead of a silently smaller gate.

LEVEL 2 — IMPRESS THEM

🔵 Medium: skipif and a strict xfail.

tests/test_skip_xfail.py

```python
import sys, pytest

@pytest.mark.skipif(sys.platform == "win32", reason="uses unix socket")
def test_unix_only(): ...

@pytest.mark.xfail(strict=True, reason="BUG-42: sort by price ignores cents for
problem_user")
def test_problem_user_price_sort(): ...
```

HIRE SIGNAL

skipif when the test doesn't apply here; a strict xfail for a known bug — and strict means a surprise pass fails the run. Run it: the empty test body passes, so the strict xfail fails.

LEVEL 3 — HERO MODE

🟣 High/Complex: an `--env` option driving settings, plus auto-skip for DB tests.

tests/conftest.py

```python
# tests/conftest.py: this `settings` fixture replaces the simple one from P03-04
import pytest
from framework.config import Settings

PROFILES = {
    "public": {},                                          # Settings defaults = public
demo apps
    "local":  {"toolshop_url": "http://localhost:4200", "toolshop_api_url":
"http://localhost:8091",
               "booker_url": "http://localhost:3001"},
    "docker": {"toolshop_url": "http://toolshop-ui:4200", "toolshop_api_url":
"http://toolshop-api:8091",
               "booker_url": "http://restful-booker:3001"},   # compose service names (P08)
}

def pytest_addoption(parser):
    parser.addoption("--env", choices=sorted(PROFILES), default="public", help="target
environment")

@pytest.fixture(scope="session")
def settings(pytestconfig):
    env = pytestconfig.getoption("--env")
    return Settings(env=env, **PROFILES[env])                  # explicit kwargs beat env
vars

def pytest_collection_modifyitems(config, items):
    if config.getoption("--env") == "public":
        skip_db = pytest.mark.skip(reason="no DB access against public demo apps; use --env
local/docker")
        for item in items:
            if "db" in item.keywords:
                item.add_marker(skip_db)
```

NOTE

Adjust the Docker hostnames to your compose service names in P08.

WHY IT'S HERO-LEVEL

One `--env` switch maps to a settings profile, and a collection hook skips DB tests where there's no database to reach.

try it Drive the conftest above + ADDED

Switch `--env` in the dropdown and watch the URLs change and the DB test skip on `public`. The last option shows what `choices=` does with a value it doesn't know.

tests/test_env_option.py

```python
# tests/test_env_option.py   (+ added: exercises the --env conftest above)
import pytest


def test_urls_follow_the_env_option(settings):
    print(f"\n  env={settings.env}  booker_url={settings.booker_url}")
    assert settings.booker_url.startswith("http")


@pytest.mark.db
def test_needs_a_database(settings):
    assert settings.env != "public"
```

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **Skip vs xfail?**

`skip` doesn't run the test because it isn't applicable in this context. `xfail` runs it but expects failure because of a known, tracked bug. It reports xfailed or xpassed, and with `strict=True` an unexpected pass fails the run so you remember to remove the mark.

H **Why enable `--strict-markers`?**

Unregistered markers are usually typos (`@pytest.mark.smok`), which silently drop tests out of `-m smoke` selections and CI gates. Strict mode turns them into errors and forces a documented, controlled marker vocabulary.

H **How do you run different test subsets in different CI stages?**

Markers plus expressions: PR gate `-m "smoke and not quarantine"`, merge/nightly `-m "not quarantine"`, DB jobs `-m db`. Combine with folder paths and `--env`. Document the mapping so developers know what runs where, and fail the job if zero tests are collected.

H **How do you add a custom command-line option to pytest?**

Implement `pytest_addoption(parser)` in the root `conftest.py` (or a plugin), calling `parser.addoption("--env", choices=[...], default=...)`, and read it via `request.config.getoption("--env")` or the `pytestconfig` fixture, typically inside a session fixture that builds settings.

SH **Why `xfail(strict=True)` by default (`xfail_strict = true`)?**

Non-strict xfails rot: the bug gets fixed, the test starts passing as XPASS, and nobody notices, so the test stays muted forever. Strict makes XPASS a failure, which forces cleanup and keeps the known-bugs list honest. Pair it with a bug-ID convention in the reason.

🤖 AI ASSIST

Ask AI to propose a marker taxonomy for a 2,000-test suite, then trim it. Fewer, well-defined markers win.

P03-06 · SUPER HERO · 4H

Hooks, plugins and coverage

App — all

Deliverable — `framework/pytest_plugin.py`, `tests/unit/test_plugin.py`

📋 The task

Build a local pytest plugin and load it from the root `conftest.py` with `pytest_plugins = ["framework.pytest_plugin"]`. The plugin should:

1. **Auto-mark** tests by folder (`tests/api/**` → `api`, `tests/ui/**` → `ui`, `tests/db/**` → `db`).
2. **Enforce policy:** `@pytest.mark.quarantine` must include `issue="..."`, otherwise it's a usage error.
3. **Record the outcome** of each phase on the test item (`item.rep_call`) so fixtures can act on failure (a screenshot-on-failure pattern for P05).
4. **Print a summary:** the 5 slowest tests and pass/fail counts per marker at the end of the run.
5. Test the plugin with `pytester`, and add `pytest-cov` on `framework/` with `--cov-fail-under=80`.

`done when` Tick these off

- [ ] The plugin tests pass (quarantine without issue → non-zero exit)
- [ ] Framework coverage is ≥80% and reported in the terminal

📘 User guide

`concepts` What you need to know

- **Hook:** a named extension point pytest calls during its lifecycle (`pytest_collection_modifyitems`, `pytest_runtest_makereport`, …).
- **Plugin:** a module implementing hooks and/or fixtures. It's loaded from `conftest.py`, `-p name`, `pytest_plugins = [...]`, or installed packages via entry points.
- **Hook wrapper:** wraps other hook implementations. Code before `yield` runs first, and after `yield` you get the result.

- **Report phases:** each test has `setup`, `call` and `teardown` reports, each with `.passed`, `.failed`, `.skipped` and `.duration`.
- `pytester`: a built-in fixture that runs pytest *inside* a test on generated files, for testing plugins.

deep dive Going deeper

The lifecycle you hook into:

```
pytest_addoption → pytest_configure → collection (pytest_generate_tests,
pytest_collection_modifyitems)
→ for each test: setup → call → teardown   (pytest_runtest_makereport after each phase)
→ pytest_terminal_summary → pytest_unconfigure
```

The "screenshot on failure" pattern combines a hook wrapper with a fixture: the hook stores each phase's report on the item (`item.rep_call`), then a fixture's teardown reads `request.node.rep_call.failed` and captures evidence. You'll reuse this in P05 and P10.

Policy as code. Enforcing "quarantine needs an issue" at collection time turns a team agreement into a failing build. That's how Super Heroes scale standards across 50 contributors without nagging in reviews.

Coverage for framework code, not tests. Measure `framework/` (shared code) with `--cov=framework`. Coverage of *tests* is meaningless, and coverage of the *app* requires instrumenting the app (a different topic). 80% line coverage of framework code with meaningful asserts (see mutation testing in P03-01) is a healthy bar.

INTERACTIVE — ONE RUN, HOOK BY HOOK, WITH THE P03-06 PLUGIN LOADED

pytest_addoption

pytest_configure

collection · pytest_generate_tests

pytest_collection_modifyitems auto-mark by folder · check quarantine

setup → makereport(setup) item.rep_setup

call → makereport(call) PASSED item.rep_call

teardown: on_failure_evidence saves nothing

makereport(teardown) item.rep_teardown

pytest_terminal_summary "QE summary" section

pytest_unconfigure

Last hook of the run.



Worked examples

LEVEL 1 — CRACK IT

🟢 Low: auto-mark by folder.

framework/pytest_plugin.py

```python
# framework/pytest_plugin.py
import pytest

LAYERS = ("api", "ui", "db", "e2e")

def pytest_collection_modifyitems(config, items):
    for item in items:
        parts = item.path.parts
        for layer in LAYERS:
            if layer in parts:
                item.add_marker(getattr(pytest.mark, layer))
```

SAY THIS

A collection hook marks tests by folder, so nobody has to remember `@pytest.mark.api`.

try it Load the plugin with `-p` + ADDED

Without the plugin, `-m api` selects nothing and pytest exits 5.

tests/api/test_layers.py

```python
# tests/api/test_layers.py   (+ added: a test that lives under tests/api/)
def test_this_test_was_auto_marked(request):
    assert request.node.get_closest_marker("api") is not None, "the plugin should have
added @pytest.mark.api"
```

LEVEL 2 — IMPRESS THEM

🔵 Medium: store phase reports and use them in a fixture.

framework/pytest_plugin.py

```python
# framework/pytest_plugin.py (continued)
@pytest.hookimpl(wrapper=True)
def pytest_runtest_makereport(item, call):
    report = yield                      # let pytest build the report
    setattr(item, f"rep_{report.when}", report)   # rep_setup / rep_call / rep_teardown
    return report

@pytest.fixture
def on_failure_evidence(request, tmp_path):
    yield
    rep = getattr(request.node, "rep_call", None)
    if rep is not None and rep.failed:
        (tmp_path / "failure.txt").write_text(f"{request.node.nodeid}\n{rep.longreprtext}")
```

NOTE

Older code uses `@pytest.hookimpl(hookwrapper=True)` with `outcome = yield; report = outcome.get_result()`. Recognise both styles.

HIRE SIGNAL

The hook wrapper stores each phase's report on the item, and a fixture's teardown reads `rep_call` to save evidence only when the test failed.

try it Evidence only on failure + ADDED

tests/test_evidence.py

```
# tests/test_evidence.py   (+ added: watch the evidence fixture react to a failure)
import pytest


@pytest.fixture
def show_evidence(tmp_path):
    yield
    # torn down AFTER on_failure_evidence (teardown is LIFO), so the file exists by now
    evidence = tmp_path / "failure.txt"
    print("\n  [evidence] " + (evidence.read_text().splitlines()[0] if evidence.exists()
else "nothing saved: the test passed"))


def test_passes(show_evidence, on_failure_evidence):
    assert 1 + 1 == 2


def test_fails_on_purpose(show_evidence, on_failure_evidence):
    assert 0.1 + 0.2 == 0.3, "floats strike again"
```

LEVEL 3 — HERO MODE

🟣 High/Complex: policy enforcement, a custom summary and a pytester test.

framework/pytest_plugin.py

```python
# framework/pytest_plugin.py: final version of the collection hook (replaces the Low example's)
from collections import Counter
import pytest

LAYERS = ("api", "ui", "db", "e2e")

def pytest_collection_modifyitems(config, items):
    offenders = []
    for item in items:
        for layer in LAYERS:                                  # 1. auto-mark by folder
            if layer in item.path.parts:
                item.add_marker(getattr(pytest.mark, layer))
        marker = item.get_closest_marker("quarantine")        # 2. enforce policy
        if marker and not marker.kwargs.get("issue"):
            offenders.append(item.nodeid)
    if offenders:
        raise pytest.UsageError("quarantine marker requires issue='...': " + ",
".join(offenders))

@pytest.hookimpl(trylast=True)
def pytest_terminal_summary(terminalreporter, exitstatus, config):
    reports = [r for key in ("passed", "failed") for r in terminalreporter.stats.get(key,
[]) if r.when == "call"]
    slowest = sorted(reports, key=lambda r: r.duration, reverse=True)[:5]
    terminalreporter.section("QE summary")
    for r in slowest:
        terminalreporter.write_line(f"{r.duration:6.2f}s  {r.nodeid}")
    by_marker = Counter(k for r in reports for k in r.keywords if k in ("api", "ui", "db",
"e2e"))
    terminalreporter.write_line(f"by layer: {dict(by_marker)}")
```

tests/unit/test_plugin.py

```python
# tests/unit/test_plugin.py
pytest_plugins = ["pytester"]

def test_quarantine_without_issue_is_rejected(pytester):
    pytester.makepyfile("""
        import pytest
        @pytest.mark.quarantine
        def test_flaky(): pass
    """)
    result = pytester.runpytest("-p", "framework.pytest_plugin")
    assert result.ret != 0
    assert "quarantine marker requires issue" in result.stdout.str() + result.stderr.str()

def test_quarantine_with_issue_runs(pytester):
    pytester.makepyfile("""
        import pytest
        @pytest.mark.quarantine(issue="QE-101")
        def test_flaky(): pass
    """)
    pytester.runpytest("-p", "framework.pytest_plugin").assert_outcomes(passed=1)
```

```
uv add --dev pytest-cov
uv run pytest tests/unit --cov=framework --cov-report=term-missing --cov-fail-under=80
```

NOTE

If `pytester` runs complain about unknown markers, register them in the generated test dir with `pytester.makeini(...)`. Debugging this is part of the exercise.

WHY IT'S HERO-LEVEL

Policy as code: a quarantine without an issue fails the run, and `pytester` proves the plugin does what it promises.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

H **What is a pytest hook? Name three you've used.**

A predefined lifecycle function pytest calls, which plugins and conftests implement to customise behaviour. Examples: `pytest_addoption` (CLI flags), `pytest_collection_modifyitems` (auto-mark, reorder, skip), `pytest_runtest_makereport` (capture results for screenshots or reporting), `pytest_generate_tests` (dynamic parametrisation), `pytest_terminal_summary` (custom summary).

H **How do you take a screenshot only when a UI test fails?**

A hook wrapper on `pytest_runtest_makereport` stores the call-phase report on the item. A fixture wrapping the page yields, then in teardown checks `request.node.rep_call.failed` and saves a screenshot/trace, attaching it to the report. (pytest-playwright also provides `--screenshot only-on-failure` and `--tracing retain-on-failure`.)

SH **How would you build and distribute a pytest plugin for multiple teams?**

Put hooks and fixtures in a package module, expose it through a `pytest11` entry point in `pyproject.toml` so it auto-loads when installed, add CLI options for opt-in behaviour, test it with `pytester` across pytest versions, version it semantically, and document the hooks and fixtures it adds. Keep it free of app-specific logic.

SH **What are the three report phases and why do they matter?**

`setup` (fixtures), `call` (the test body) and `teardown`. A failure in setup is an *error*, not a *failure*. It usually means environment or data problems, not a product bug, which is critical for triage. Reporting and screenshot logic must check the right phase (usually `call`), and teardown errors (cleanup failed) should be visible but separated.

H **What does code coverage tell you about a test framework, and what doesn't it tell you?**

It shows which lines ran during tests, so it finds untested framework code. It doesn't show whether behaviour was *verified* (weak asserts still give 100%), nor anything about product coverage. Combine it with mutation testing and requirement/risk coverage mapping.

🤖 AI ASSIST

Ask AI to list pytest hooks relevant to reporting and explain when each fires, then verify with a tiny plugin that prints in each.

🏁 PHASE CHECKPOINT — P03

Prove it without help

⏱️ Self-test (30 min, no AI)

write from memory a conftest with a session HTTP fixture, a factory fixture with cleanup, a `--env` option, and one parametrized test with ids.

- [ ] Done from memory, without AI, inside 30 minutes

30-MINUTE TIMER

30:00

mock interview

AI mock interviewer prompt

```
Interview me on pytest for an SDET role. Cover fixtures & scopes, conftest,
parametrize, markers, skip vs xfail, hooks and plugins. Ask 8 questions one at a
time, increasing difficulty, and include one "debug this conftest" scenario.
Score each answer 1-5 with what a senior answer would add.
```

Record your results in `interview-notes/P03.md`.

BEYOND P03 — + ADDED

🧩 Five gaps worth closing

P03 covers the pytest you need every day. These five come up in interviews and in real framework code soon after, and each one runs here.

01 Mocking: fakes you can configure and inspect

Unit tests for **framework code** (clients, helpers, plugins) shouldn't need a live API. A `Mock` answers whatever you configure, can raise on demand, and records how it was called. `monkeypatch` is pytest's own way to swap a name for one test.

tests/unit/test_mocking.py

```python
# tests/unit/test_mocking.py   (+ added)
from unittest.mock import Mock, patch

import pytest


def api_is_healthy(session, base_url: str) -> bool:
    """Framework code under test: true when /ping answers 201."""
    return session.get(f"{base_url}/ping", timeout=5).status_code == 201


def test_healthy_when_ping_returns_201():
    session = Mock()
    session.get.return_value.status_code = 201              # configure the fake's answer
    assert api_is_healthy(session, "http://booker")
    session.get.assert_called_once_with("http://booker/ping", timeout=5)   # and check how
it was called


def test_unhealthy_on_server_error():
    session = Mock()
    session.get.return_value.status_code = 503
    assert not api_is_healthy(session, "http://booker")


def test_a_timeout_propagates():
    session = Mock()
    session.get.side_effect = TimeoutError("read timed out")   # make the fake raise
    with pytest.raises(TimeoutError):
        api_is_healthy(session, "http://booker")


def test_patch_replaces_a_name_for_one_block():
    with patch("time.time", return_value=0):                   # patch where it is looked up
        import time
        assert time.time() == 0


def test_monkeypatch_is_the_pytest_way(monkeypatch):
    monkeypatch.setattr("os.getcwd", lambda: "/pretend")      # undone automatically after
the test
    import os
    assert os.getcwd() == "/pretend"
```

H

Mock vs stub vs fake?

A **stub** returns canned answers; a **mock** also records calls so you can assert on them; a **fake** is a working lightweight implementation (like this page's simulated Restful-Booker). Patch where a name is **looked up**, not where it's defined, and don't mock what you're testing.

02 Parametrized fixtures and indirect parametrize

P03-03 parametrizes *tests*. You can also parametrize a *fixture* — every test that uses it runs once per value — or route a parametrize value through a fixture with `indirect=True`, so the fixture builds the real object.

tests/unit/test_fixture_params.py

```
# tests/unit/test_fixture_params.py   (+ added)
import pytest


@pytest.fixture(params=["chromium", "firefox"], ids=str)
def browser_name(request):
    return request.param                        # every test using this fixture runs once
per param


def test_runs_on_each_browser(browser_name):
    assert browser_name in {"chromium", "firefox"}


@pytest.fixture
def user(request):
    return {"username": request.param, "password": "secret_sauce"}   # build an object from
the raw value


@pytest.mark.parametrize("user", ["standard_user", "locked_out_user"], indirect=True)
def test_indirect_parametrize(user):
    assert user["password"] == "secret_sauce"
```

03 Output, logs and warnings

`capsys` captures `print`, `caplog` captures logging, and `pytest.warns` asserts a warning — useful when your framework deprecates a helper.

tests/unit/test_capture.py

```
# tests/unit/test_capture.py   (+ added)
import logging
import warnings

import pytest


def legacy_login(user: str) -> str:
    warnings.warn("legacy_login is deprecated, use LoginPage.login", DeprecationWarning,
stacklevel=2)
    print(f"logging in {user}")
    logging.getLogger("framework").info("login attempt for %s", user)
    return user


def test_prints_are_captured(capsys):
    legacy_login("standard_user")
    assert "logging in standard_user" in capsys.readouterr().out


def test_logs_are_captured(caplog):
    caplog.set_level(logging.INFO)
    legacy_login("standard_user")
    assert "login attempt for standard_user" in caplog.text


def test_warning_is_raised():
    with pytest.warns(DeprecationWarning, match="deprecated"):
        legacy_login("standard_user")
```

04 Ask pytest what it sees

Before debugging a fixture or marker, ask pytest: `--co` lists what would run, `--setup-plan` shows the fixture plan without running anything, `--markers` lists registered markers, and `--fixtures-per-test` shows where each fixture comes from.

tests/unit/test_introspect.py

```python
# tests/unit/test_introspect.py   (+ added: something for pytest to describe)
import pytest


@pytest.fixture(scope="session")
def settings():
    return {"env": "public"}


@pytest.fixture
def booking(settings):
    yield {"id": 1}


@pytest.mark.smoke
def test_booking_has_an_id(booking):
    assert booking["id"] == 1
```

05 Which config file wins

pytest picks **one** config file. It looks upward from the common folder of your test paths and takes the first match in this order; the header line `configfile: …` tells you which one it found.

ORDER	FILE	NOTE
1	`pytest.ini` / `.pytest.ini`	wins even if it's empty
2	`pyproject.toml`	only if it has `[tool.pytest.ini_options]` — what this repo uses
3	`tox.ini`	with a `[pytest]` section
4	`setup.cfg`	with a `[tool:pytest]` section

H **A setting in pyproject.toml is ignored. Why?**

Another config file was found first (often a stray `pytest.ini`), or pytest ran from a folder whose rootdir is different. Run `pytest --co -q` and read the `rootdir:` and `configfile:` lines in the header; pass `-c` to force a file.

FIELD GUIDE

🎓 Pattern recognition

The situation → the pytest tool to reach for.

`same check, many inputs`

parametrize + ids

each case reported on its own

`expensive, read-only setup`

session fixture

created once, shared safely

`a test needs N resources`

factory fixture

track IDs, clean up LIFO

`passes alone, fails in the suite`

shared state

mutable defaults, globals, leftover data

`CI is green but ran nothing`

exit code 5 + --strict-markers

fail the job on 5

`a known, tracked bug`

xfail(strict=True)

alerts you when it's fixed

`not applicable in this env`

skipif / collection hook

with a reason

`need evidence only on failure`

makereport wrapper + fixture

read rep_call in teardown

`a team rule nobody remembers`

a hook in a plugin

policy as code

THE GOLDEN RULE

A test you have never seen fail is not a test yet.
Break the code, watch it go red, then trust the green.

Built from *learning-path/phases/P03-pytest.md* · QE learning path, zero → hero

🐍 next: P04 — API testing with requests, on top of these fixtures